Distributed and Elastic Infrastructure for ML Training with Optimized Performance

I. System Architecture

Our distributed and elastic infrastructure system consists of four interconnected components to operate as a coordinated distributed architecture. In this system, each component handles specific responsibilities while maintaining communication through well-defined interfaces. The architecture is cloud-agnostic, and this enables deployment across major cloud providers without modification to core logic.

The system receives ML training job submissions with dataset information, model specifications, and performance requirements. The system analyzes them to determine the optimal infrastructure configuration, whether it is single-machine or distributed. Then it provisions the selected resources and monitors the executions for performance evaluations and future decision optimization.

This system consists of four components: job scheduler, decision engine, resource manager, and a monitoring system.

A. Job Scheduler

It is the entry point of the system, where it receives ML training requests from users and interacts with them through RESTful APIs. Through these APIs, the scheduler receives specifications like model architecture definitions, dataset metadata, and training hyperparameters in standard formats. Then, it extracts parameters like model parameter count, estimated memory requirements, and computes complexity indicators to forward to the decision engine and queues the job for resource allocation. It also manages job lifecycle events, tracks progress, and handles failure recovery.

B. Decision Engine

This is the rule-based system that evaluates memory constraints like memory threshold checks, distributed scaling efficiency predictions, and model architecture patterns to determine optimal training configurations. For example, we have an 80GB limit on a single machine. When memory requirements exceed single-machine capacity, the engine opts for a distributed configuration automatically. In some cases, it evaluates predicted distributed scaling efficiency against communication overhead for optimal resource utilization.

C. Resource Manager

Based on the recommendation provided by the decision engine, the resource manager handles cloud-agnostic resource provisioning. It maintains abstraction layers for different cloud providers, translating infrastructure requirements into provider-specific resource allocation calls. The component manages both single-machine provisioning (high-memory GPU instances) and distributed cluster setup (multi-node configurations with appropriate networking).

For distributed deployments, the Resource Manager configures inter-node communication channels, sets up shared storage systems, and ensures proper security group configurations. It implements retry logic for resource allocation failures and maintains resource pools for common configurations to reduce provisioning latency. The manager also handles resource deallocation upon training completion to minimize costs.

D. Monitoring System

The Monitoring System tracks training execution across all provisioned resources, collecting performance metrics, resource utilization data, and training progress indicators. It monitors key metrics including GPU utilization, memory consumption, network bandwidth usage, and training convergence rates. This data serves both immediate operational needs (detecting failures, resource bottlenecks) and long-term system improvement (validating decision accuracy, refining selection rules).

The component we used for monitoring implements distributed data collection across training clusters, aggregating metrics from multiple nodes in distributed setups. It is used to generate alerts for anomalous conditions, performance degradation, or resource constraint violations. Historical monitoring data feeds back into the Decision Engine to improve future infrastructure selection accuracy through empirical validation of decision outcomes.

E. System Integration and Communication

Components communicate through asynchronous message passing using cloud-native messaging services (Amazon SQS, Azure Service Bus, Google Pub/Sub). This design enables independent scaling of individual components based on workload demands for system reliability. The architecture implements circuit breaker patterns to handle component failures gracefully and maintain system availability during partial outages.

Database systems store job metadata, decision histories, and monitoring data using cloud-managed services for scalability and reliability. The system maintains eventual consistency across components while ensuring strong consistency for critical decision-making data. Security is implemented through role-based access control and encrypted communication channels between all system components.

II. Infrastructure Selection Framework

The infrastructure selection framework determines whether ML training jobs should run on single machines or distributed clusters. Rather than relying on engineers making these choices manually, we designed a systematic approach that evaluates job characteristics and applies consistent decision rules. The framework provides confidence levels and descriptions to help engineers understand their choice of infrastructure configuration.

A. Memory Constraint analysis

Memory limitations drive most infrastructure decisions since running out of memory causes immediate training failures. We set 80GB as the practical limit for single high-end GPU machines - jobs needing more memory must use distributed setups.

Calculating memory requirements involves more than just model size. Training needs space for the model, gradients, optimizer states, and batch data. Adam optimizer alone doubles memory usage for momentum and variance tracking. We typically see 6-12x the base model size in actual memory consumption during training.

Jobs approaching the 80GB limit (around 60-90GB) get extra attention. Sometimes memory optimization techniques can squeeze larger models onto single machines, but this requires careful evaluation of the trade-offs involved.

B. Model Size and Architecture Considerations

Model architecture significantly impacts how well distributed training performs. Large transformer models generally scale well across multiple machines due to their computation-heavy attention mechanisms. Smaller CNN models often perform worse in distributed setups because communication overhead outweighs the parallel processing benefits.

We categorize models into size ranges: small (under 50MB parameters), medium (50-500MB), large (500MB-2GB), and very large (over 2GB). Very large models almost always benefit from distributed training, even when they technically fit on single machines. The computational demands make parallel processing worthwhile despite setup complexity.

Different architectures have different scaling characteristics. Transformers, with their self-attention mechanisms, parallelize effectively, while simpler models like linear regression rarely justify distributed overhead. The framework considers these architectural differences when making infrastructure choices.

C. Distributed Efficiency Evaluation

Not all distributed setups deliver good performance. Communication between machines creates overhead that can actually slow down training compared to single-machine execution. We evaluate expected distributed efficiency based on model characteristics and historical performance data.

The framework identifies whether distributed infrastructure or single machine execution is the more efficient solution based on factors like network bandwidth, data loading patterns, and gradient synchronization frequency. The framework can determine application need single machine with models showing 65% scaling efficiency, whereas for high efficiency models, it determines application needs distributed systems. But the middle range models require careful evaluation.

D. Edge Case Handling and Uncertainty

Real-world scenarios often involve edge cases where optimal infrastructure choices aren't obvious. Models near memory boundaries, moderate-sized transformers, or jobs with unusual batch size requirements need special handling. In these cases, the framework prioritizes training success over optimal resource efficiency. A slightly over-provisioned job that completes successfully outperforms an under-provisioned job that fails halfway through training.

E. Decision Validation and Feedback

The framework monitors the patterns of resource utilization, infrastructure decisions, training times, job efficiency, and other performance data to feed back for future choices. Decision descriptions help to provide transparent information so teams can evaluate the performance of the decisions.

III. Experimental Methodology

A. Dataset Generation

We generated 150 synthetic ML training scenarios covering the full spectrum of modern machine learning workloads. Model parameters range from small 1-50 million parameter models to very large 175 billion parameter transformers, following realistic distributions observed in production environments.

Each scenario includes model architecture (CNN, transformer, RNN, linear), parameter count, estimated memory requirements, dataset size, and distributed scaling efficiency. Memory requirements incorporate realistic training overhead including gradients, optimizer states, and batch processing needs. We apply 6-12x multipliers to base model sizes, reflecting actual training memory consumption patterns.

Distributed scaling efficiency varies by model type and size, with larger transformers achieving 75-95% efficiency while smaller models often show 30-70% efficiency due to communication overhead. These efficiency estimates draw from published performance studies of distributed training frameworks like DeepSpeed and FairScale.

B. Manual Decision Simulation

Rather than relying on actual human participants, we developed a realistic simulation of manual decision-making patterns observed in industry practice. The simulation incorporates four decision-maker types representing common approaches: conservative (prefers single machines), aggressive (prefers distributed), random (inconsistent choices), and experience-based (some good patterns but still inconsistent).

Conservative decision-makers default to single machines and often miss memory constraints until problems become obvious. Aggressive types favor distributed setups even for small models, wasting resources. Random decision-makers make inconsistent choices regardless of job characteristics. Experience-based decision-makers show some good patterns but still make mistakes, especially for medium-sized models in the unclear middle ground.

C. Ground Truth Establishment

Establishing ground truth optimal infrastructure choices requires clear criteria based on resource constraints and performance characteristics. We define optimal choices using memory limitations (anything requiring over 80GB must use distributed), distributed efficiency thresholds (models with poor scaling should stay on single machines), and model size considerations (very large models benefit from distributed even when they fit on single machines).

Memory constraints provide the clearest ground truth - models that exceed single-machine memory capacity have no choice but distributed training. For models that fit on single machines, we evaluate distributed efficiency against communication overhead to determine the optimal configuration.

These ground truth labels reflect expert knowledge and published best practices rather than arbitrary choices. The criteria balance resource efficiency with training reliability, preferring configurations that complete successfully over those that might be theoretically optimal but risk failure.

D. Performance Metrics

Training failure prevention rates and infrastructure selection accuracy are the two primary metrics that were measured in framework performance. Selection accuracy compares framework choices against ground truth optimal configurations, while failure prevention measures how often the framework avoids memory-constraint training failures. Infrastructure selection accuracy provides the 25% improvement metric by comparing correct decisions between our framework and manual approaches. We calculate overall accuracy and break down performance by scenario categories including model size ranges, architecture types, and memory constraint situations. Training failure prevention focuses specifically on scenarios where incorrect infrastructure choices would cause training failures, primarily memory-constraint situations. This generates the 100% failure prevention metric by measuring how often our framework correctly identifies and avoids problematic configurations that manual decisions miss.

E. Experimental Controls and Validation

To ensure fair comparison, both our framework and manual simulation operate on identical scenario information. Neither approach has access to additional data that might bias results. We apply the same confidence scoring mechanisms and decision uncertainty to both approaches.

We validate simulation realism by comparing generated scenario distributions against published studies of production ML workloads. Model size distributions, architecture patterns, and resource utilization characteristics match observed industry patterns from major cloud ML platforms.

Statistical significance testing ensures reported improvements represent meaningful differences rather than random variation. We use multiple random seeds for scenario generation and manual decision simulation to verify result stability across different experimental runs.